

# ESP32-S3 "AEDI" Radar Node: LED Visual Interface Specification

This document outlines the complete visual language and physical layout for a single-pixel (WS2812B/NeoPixel) RGB LED, designed to communicate complex radar, network, and system states without a screen.

## I. The Physical Layout (Hardware Diffusion)

A bare LED is harsh and directional. To make it look like a premium piece of hardware, these diffusion techniques change how the light disperses.

- **The Frosted "Eye" (Soft Diffusion):** \* **Design:** Place a small dome of frosted acrylic, milky white PLA (3D printed), or hot glue directly over the LED.
  - **Effect:** Transforms the harsh pinpoint of light into a soft, glowing "orb" that looks professional and ambient.
- **The Edge-Lit Radar Wave (Directional):** \* **Design:** Mount a small, clear piece of acrylic vertically right above the LED with a laser-etched design (like concentric WiFi radar waves).
  - **Effect:** The light shoots up through the clear plastic and brilliantly illuminates only the etched lines.
- **The Light Pipe Halo (Cylindrical):** \* **Design:** Use a curved piece of clear plastic tubing (a light pipe) wrapped around the inside edge of a cylindrical case, starting at the LED.
  - **Effect:** Carries the light in a circle, making your single LED look like an illuminated "halo" ring around the device.

## II. System Lifecycle & Critical States

These are top-level hardware overrides that indicate the physical health of the ESP32.

- **Boot Sequence ("Prism Sweep")**
  - **Trigger:** Device powers on or completes a reboot.
  - **Look:** Rapid, fluid gradient sweep through the spectrum: Red → Yellow → Green → Cyan → Blue → Magenta → White.
  - **Timing:** 1.5 seconds total, ending with a 500ms maximum-brightness White flash.
- **Connection Established ("System Healthy")**
  - **Trigger:** Successfully connecting to the WiFi mesh, locking onto the Pi Zero, and verifying the AI pipeline is ready.
  - **Look:** A smooth fade into a solid, bright **Emerald Green**.
  - **Timing:** Holds for exactly 2 seconds, followed by a crisp double-blink, then drops into the Blue Baseline. This is the ultimate "All Systems Go" indicator.

- **Hardware Crash ("Fatal Fault")**
  - **Trigger:** Watchdog timer reset, memory leak, or brownout.
  - **Look:** Chaotic, arrhythmic strobe of pure **#FF0000** (Red) at absolute maximum brightness.
  - **Timing:** Random intervals (e.g., 50ms spike, 200ms off, two 20ms micro-flashes) to mimic a broken circuit.
- **OTA Update / Matrix Upload ("Do Not Unplug")**
  - **Trigger:** Firmware flashing or downloading new ONNX AI models.
  - **Look:** High-speed, hypnotic oscillation between Cyan and Magenta.
  - **Timing:** Smooth crossfade taking ~200ms per color. Visually busy to warn the user against unplugging power.

### III. Network Health, Roles & Signal Strength

Periodic diagnostic checks that interrupt normal activity to report WiFi mesh health and network topology.

- **The 10-Second Signal Check ("Yellow Beacon")**
  - **Trigger:** Fires exactly once every 10 seconds.
  - **Duration:** Lasts exactly 1 second, then resumes previous activity.
  - **100% - 70% RSSI (Strong):** Sharp, aggressive spike to maximum bright Yellow. Holds for 200ms, sharply fades out.
  - **69% - 30% RSSI (Moderate):** "Lazy" Yellow swell. Slow fade up to 50% brightness, slow fade down.
  - **29% - 1% RSSI (Weak):** Dim, ghostly Yellow shimmer. Caps at 10% brightness with a slight flicker, like a dying bulb.
  - **0% RSSI (Disconnected):** Replaces Yellow with a harsh, dim Red double-flash.
- **Environmental Interference ("The Orange Warning")**
  - **Trigger:** The radar detects excessive 2.4GHz/5GHz noise (e.g., a microwave, heavy WiFi congestion) or physical obstruction blocking the CSI multipath reflections.
  - **Look:** A slow, continuous throb in **Burnt Orange** (**#FF4500**).
  - **Timing:** Overrides the Blue Baseline idle state until the interference is cleared or the environment recalibrates. It acts as a clear "I cannot see properly" warning.
- **The 20-Second Role Check ("Node Identity Ping")**
  - **Trigger:** Fires exactly once every 20 seconds (staggered to avoid overlapping the 10-second Yellow Beacon).
  - **Purpose:** Visually maps the physical topology of the mesh network.
  - **Receiving Node (Hub/Sink):** A smooth, double-ripple in **Deep Violet** or **Indigo**.
  - **Transmitter Nodes (Edge Sensors):** A crisp, singular blink in **Teal** or **White**.
- **Mesh Node Stutter (Optional Overlay)**
  - **Look:** Micro-stutters injected at the peak of the Yellow Beacon pulse.
  - **Meaning:** 3 quick stutters at the peak means the ESP32 is currently talking to 3 other nodes in the mesh.

## IV. Radar Sensing & AI Activity

The core visual language representing what the CSI data and ONNX models are actively doing.

- **Radar Calibration ("Baseline Sweep")**
  - **Trigger:** First boot in a new room; mapping static walls/furniture.
  - **Look:** Slow, methodical expanding White pulse.
  - **Timing:** Pulses from 0% → 10%, resets. 0% → 20%, resets. Acts like a sonar ping getting louder until it hits 100%.
- **Idle / Scanning ("The Blue Baseline")**
  - **Trigger:** Default state. System is **Online and Healthy**, gathering CSI data, no humans currently detected.
  - **Look:** Dim, rhythmic "breathing" effect in **Deep Blue**.
  - **Timing:** Speed maps to data throughput. Fast, shallow pulses = high data rate. Slow, deep ocean pulses = low power/slow data rate.
- **Human Detected ("Contact Pulse")**
  - **Trigger:** AI model scores a positive hit on human presence or movement.
  - **Look:** Warm Amber or Soft Gold double "heartbeat" pulse.
  - **Timing:** *Thump-thump... pause... thump-thump*. Can be dynamically synced to actual detected respiration or heart rate via mmWave/CSI.
- **Heavy AI Processing ("Synapse Spark")**
  - **Trigger:** Pi Zero/ESP32 is crunching heavy matrix math or MinCut algorithms.
  - **Look:** Rapid, erratic flickering in Magenta or Purple.
  - **Timing:** Looks like a tiny electrical storm or processing cluster working at maximum load.

## V. Advanced Pulsing Dynamics (Modifiers)

These aren't standalone states, but behaviors applied to the pulses above to convey extra data.

- **The Sonar Tail (Latency Check)**
  - **Concept:** Alter the "decay" (fade-out speed) of the Yellow Signal pulse based on network ping/latency.
  - **Fast Fade:** Low latency (instant response).
  - **Long, Dragging Fade:** High latency (laggy network).

BASIS:

```
#include <FastLED.h>
```

- 
- `// --- HARDWARE CONFIGURATION ---`
- `#define NUM_LEDS 1`

```

○ #define DATA_PIN 48    // Change this to your ESP32-S3's built-in LED pin
  (often 48, 8, or 38)
○ #define BRIGHTNESS 255
○ #define LED_TYPE WS2812B
○ #define COLOR_ORDER GRB
○
○ CRGB leds[NUM_LEDS];
○
○ // --- SYSTEM STATES ---
○ enum SystemState {
○   BOOT_SWEEP,
○   CONN_ESTABLISHED,
○   CALIBRATING,
○   IDLE_SCANNING,
○   HUMAN_DETECTED,
○   AI_PROCESSING,
○   INTERFERENCE,
○   OTA_UPDATE,
○   CRASH_FAULT
○ };
○
○ // --- GLOBAL VARIABLES ---
○ SystemState currentState = BOOT_SWEEP;
○ bool isHubNode = false;    // Set to true for Deep Violet receiving node,
  false for Teal edge node
○ int currentRSSI = 85;      // 0-100 scale. Update this variable from your
  WiFi logic
○ unsigned long stateStartTime = 0;
○
○ // Overlay Timers
○ unsigned long lastYellowBeacon = 0;
○ unsigned long lastRolePing = 0;
○
○ void setup() {
○   Serial.begin(115200);
○   FastLED.addLeds<LED_TYPE, DATA_PIN, COLOR_ORDER>(leds,
    NUM_LEDS).setCorrection(TypicalLEDStrip);
○   FastLED.setBrightness(BRIGHTNESS);
○
○   // Initialize state
○   stateStartTime = millis();
○ }
○
○ void loop() {
○   unsigned long currentMillis = millis();
○   CRGB baseColor = CRGB::Black;
○
○   // -----

```

```

○ // 1. BASE LEVEL STATES (Radar & AI Activity)
○ // -----
○ switch (currentState) {
○   case BOOT_SWEEP:
○     baseColor = renderBootSweep(currentMillis);
○     break;
○   case CONN_ESTABLISHED:
○     baseColor = renderConnectionEstablished(currentMillis);
○     break;
○   case CALIBRATING:
○     baseColor = renderCalibrating(currentMillis);
○     break;
○   case IDLE_SCANNING:
○     baseColor = renderIdleScanning(currentMillis);
○     break;
○   case HUMAN_DETECTED:
○     baseColor = renderHumanDetected(currentMillis);
○     break;
○   case AI_PROCESSING:
○     baseColor = renderAIProcessing(currentMillis);
○     break;
○   case INTERFERENCE:
○     baseColor = renderInterference(currentMillis);
○     break;
○   case OTA_UPDATE:
○     baseColor = renderOTAUpdate(currentMillis);
○     break;
○   case CRASH_FAULT:
○     baseColor = renderCrashFault(currentMillis);
○     break;
○ }
○
○ // -----
○ // 2. OVERLAY LEVEL (Network Health & Roles)
○ // These override the base state periodically.
○ // -----
○
○ // A. 10-Second Yellow Beacon (Overrides for 1 second)
○ if (currentMillis - lastYellowBeacon >= 10000) {
○   lastYellowBeacon = currentMillis;
○ }
○ unsigned long beaconElapsed = currentMillis - lastYellowBeacon;
○ if (beaconElapsed < 1000 && currentState != CRASH_FAULT &&
    currentState != BOOT_SWEEP && currentState != OTA_UPDATE) {
○   baseColor = renderYellowBeacon(beaconElapsed, currentRSSI);
○ }
○
○

```

```

○ // B. 20-Second Role Check (Overrides for 1 second, offset by 5s to avoid
  overlap)
○ if (currentMillis - lastRolePing >= 20000) {
○   lastRolePing = currentMillis;
○ }
○ unsigned long rolePingElapsed = currentMillis - lastRolePing;
○ // Offset execution so it doesn't fire at the exact same time as the 10s
  beacon
○ if (rolePingElapsed > 5000 && rolePingElapsed < 6000 && currentState !=
  CRASH_FAULT && currentState != BOOT_SWEEP && currentState !=
  OTA_UPDATE) {
○   baseColor = renderRolePing(rolePingElapsed - 5000, isHubNode);
○ }
○
○ // -----
○ // 3. RENDER
○ // -----
○ leds[0] = baseColor;
○ FastLED.show();
○
○ // --- DEMO STATE CYCLING (Remove in production) ---
○ // simulateRadarEvents(currentMillis);
○ }
○
○ //
  =====
  =====
○ // ANIMATION RENDERING FUNCTIONS
○ //
  =====
  =====
○
○ CRGB renderBootSweep(unsigned long currentMillis) {
○   unsigned long elapsed = currentMillis - stateStartTime;
○   if (elapsed < 1500) {
○     // Fast rainbow cycle
○     uint8_t hue = (elapsed * 255) / 1500;
○     return CHSV(hue, 255, 255);
○   } else if (elapsed < 2000) {
○     // 500ms White Flash
○     return CRGB::White;
○   } else {
○     // Transition to Connection Established
○     setSystemState(CONN_ESTABLISHED);
○     return CRGB::Black;
○   }
○ }
○ }
○

```

```

○ CRGB renderConnectionEstablished(unsigned long currentMillis) {
○   unsigned long elapsed = currentMillis - stateStartTime;
○   if (elapsed < 1500) {
○       // Solid bright Emerald Green
○       return CRGB(0, 255, 50);
○   } else if (elapsed < 1600 || (elapsed > 1700 && elapsed < 1800)) {
○       // Crisp double blink
○       return CRGB(0, 255, 50);
○   } else if (elapsed < 2000) {
○       return CRGB::Black;
○   } else {
○       setSystemState(IDLE_SCANNING);
○       return CRGB::Black;
○   }
○ }
○
○ CRGB renderIdleScanning(unsigned long currentMillis) {
○   // Deep Blue Baseline - "Breathing" effect
○   // beatsin8(BPM, min_brightness, max_brightness)
○   uint8_t breath = beatsin8(15, 30, 180);
○   return CHSV(160, 255, breath); // 160 is Deep Blue
○ }
○
○ CRGB renderHumanDetected(unsigned long currentMillis) {
○   // Warm Amber Double Heartbeat
○   unsigned long beatTime = currentMillis % 1500; // 1.5s loop
○   CRGB amber = CHSV(32, 255, 255); // Warm Amber
○
○   if (beatTime < 150) return amber;
○   else if (beatTime < 300) return CRGB::Black;
○   else if (beatTime < 450) return amber;
○   else return CRGB::Black; // Pause for the rest of the 1500ms
○ }
○
○ CRGB renderAIProcessing(unsigned long currentMillis) {
○   // Rapid erratic flickering in Magenta/Purple
○   if (random8() > 100) {
○       return CHSV(192, 255, random8(100, 255)); // Magenta varying brightness
○   }
○   return CRGB::Black;
○ }
○
○ CRGB renderInterference(unsigned long currentMillis) {
○   // Burnt Orange slow throb
○   uint8_t throb = beatsin8(20, 50, 255);
○   return CRGB(255, 69, 0).nscale8(throb);
○ }
○

```

```

○ CRGB renderCalibrating(unsigned long currentMillis) {
○   // Expanding White Pulse
○   unsigned long cycleTime = currentMillis % 2000; // 2s expanding cycle
○   uint8_t brightness = map(cycleTime, 0, 1999, 0, 255);
○   return CRGB::White.nscale8(brightness);
○ }
○
○ CRGB renderOTAUpdate(unsigned long currentMillis) {
○   // Hypnotic Cyan and Magenta oscillation
○   uint8_t wave = beatsin8(150, 0, 255); // Fast wave
○   return blend(CRGB::Cyan, CRGB::Magenta, wave);
○ }
○
○ CRGB renderCrashFault(unsigned long currentMillis) {
○   // Chaotic Red Strobe
○   if (random8() > 180) {
○     return CRGB::Red;
○   }
○   return CRGB::Black;
○ }
○
○ // --- OVERLAY RENDERERS ---
○
○ CRGB renderYellowBeacon(unsigned long elapsed, int rssi) {
○   CRGB yellow = CRGB::Yellow;
○
○   if (rssi == 0) {
○     // Disconnected: Red double-flash
○     if (elapsed < 100 || (elapsed > 200 && elapsed < 300)) return CRGB::Red;
○     return CRGB::Black;
○   }
○   else if (rssi >= 70) {
○     // Strong: Sharp aggressive spike
○     if (elapsed < 200) return yellow; // Hold max for 200ms
○     else if (elapsed < 600) return yellow.fadeToBlackBy(map(elapsed, 200,
600, 0, 255)); // Sharp fade
○     return CRGB::Black;
○   }
○   else if (rssi >= 30) {
○     // Moderate: Lazy swell (up and down)
○     uint8_t wave = sin8(map(elapsed, 0, 1000, 0, 255));
○     return yellow.nscale8(map(wave, 0, 255, 0, 128)); // Caps at 50%
brightness
○   }
○   else {
○     // Weak: Ghostly shimmer
○     if (random8() > 128) return yellow.nscale8(25); // ~10% brightness flicker
○     return CRGB::Black;
○   }
○ }

```

```

○ }
○ }
○
○ CRGB renderRolePing(unsigned long elapsed, bool isHub) {
○   if (isHub) {
○     // Hub Node: Deep Violet Double Ripple
○     CRGB violet = CRGB::DarkViolet;
○     if (elapsed < 200) return violet.nscale8(map(elapsed, 0, 200, 0, 255));
○     else if (elapsed < 400) return violet.nscale8(map(elapsed, 200, 400, 255,
○     0));
○     else if (elapsed < 600) return violet.nscale8(map(elapsed, 400, 600, 0,
○     255));
○     else if (elapsed < 800) return violet.nscale8(map(elapsed, 600, 800, 255,
○     0));
○     return CRGB::Black;
○   } else {
○     // Edge Node: Crisp singular Teal blink
○     if (elapsed > 400 && elapsed < 500) return CRGB::Teal;
○     return CRGB::Black;
○   }
○ }
○
○ //
○ =====
○ =====
○ // HELPER FUNCTIONS
○ //
○ =====
○ =====
○
○ void setSystemState(SystemState newState) {
○   if (currentState != newState) {
○     currentState = newState;
○     stateStartTime = millis();
○   }
○ }
○
○ // --- OPTIONAL: Demo Function to test states without radar logic ---
○ void simulateRadarEvents(unsigned long currentMillis) {
○   static unsigned long lastStateChange = 0;
○
○   if (currentMillis - lastStateChange > 15000) { // Change state every 15s
○     lastStateChange = currentMillis;
○     if (currentState == IDLE_SCANNING)
○       setSystemState(HUMAN_DETECTED);
○     else if (currentState == HUMAN_DETECTED)
○       setSystemState(AI_PROCESSING);

```

- else if (currentState == AI\_PROCESSING)  
       setSystemState(INTERFERENCE);
- else setSystemState(IDLE\_SCANNING);
- }
- }
- 

Examples:

```
#include <FastLED.h>
#include <WiFi.h>

// --- WIFI CONFIGURATION ---
const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";

// --- HARDWARE CONFIGURATION ---
#define NUM_LEDS 1
#define DATA_PIN 48     // Change this to your ESP32-S3's built-in LED pin
#define BRIGHTNESS 255
#define LED_TYPE WS2812B
#define COLOR_ORDER GRB

CRGB leds[NUM_LEDS];

// --- SYSTEM STATES ---
enum SystemState {
  BOOT_SWEEP,
  CONN_ESTABLISHED,
  CALIBRATING,
  IDLE_SCANNING,
  HUMAN_DETECTED,
  AI_PROCESSING,
  INTERFERENCE,
  OTA_UPDATE,
  CRASH_FAULT
};

// --- GLOBAL VARIABLES ---
SystemState currentState = BOOT_SWEEP;
bool isHubNode = false;     // Set to true for Deep Violet receiving node, false for Teal
edge node
int currentRSSI = 0;         // 0-100 scale. Now updated automatically via WiFi.RSSI()
unsigned long stateStartTime = 0;

// Overlay Timers
unsigned long lastYellowBeacon = 0;
unsigned long lastRolePing = 0;
unsigned long lastWiFiCheck = 0;
```

```

unsigned long lastRadarMockEvent = 0;

void setup() {
  Serial.begin(115200);

  // Initialize LED
  FastLED.addLeds<LED_TYPE, DATA_PIN, COLOR_ORDER>(leds,
NUM_LEDS).setCorrection(TypicalLEDStrip);
  FastLED.setBrightness(BRIGHTNESS);

  // Initialize WiFi (Non-blocking mode)
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  // Initialize state
  stateStartTime = millis();
  Serial.println("System Booting...");
}

void loop() {
  unsigned long currentMillis = millis();
  CRGB baseColor = CRGB::Black;

  // 1. RUN ACTUAL SYSTEM LOGIC (WiFi, RSSI, and Radar)
  manageSystemLogic(currentMillis);

  // 2. BASE LEVEL STATES (Animations)
  switch (currentState) {
    case BOOT_SWEEP:
      baseColor = renderBootSweep(currentMillis);
      break;
    case CALIBRATING: // Using calibrating as our "Attempting WiFi Connection" state
      baseColor = renderCalibrating(currentMillis);
      break;
    case CONN_ESTABLISHED:
      baseColor = renderConnectionEstablished(currentMillis);
      break;
    case IDLE_SCANNING:
      baseColor = renderIdleScanning(currentMillis);
      break;
    case HUMAN_DETECTED:
      baseColor = renderHumanDetected(currentMillis);
      break;
    case AI_PROCESSING:
      baseColor = renderAIProcessing(currentMillis);
      break;
    case INTERFERENCE:
      baseColor = renderInterference(currentMillis);

```

```

        break;
    case OTA_UPDATE:
        baseColor = renderOTAUpdate(currentMillis);
        break;
    case CRASH_FAULT:
        baseColor = renderCrashFault(currentMillis);
        break;
}

// 3. OVERLAY LEVEL (Network Health & Roles)

// A. 10-Second Yellow Beacon (Overrides for 1 second)
if (currentMillis - lastYellowBeacon >= 10000) {
    lastYellowBeacon = currentMillis;
}
unsigned long beaconElapsed = currentMillis - lastYellowBeacon;
if (beaconElapsed < 1000 && currentState != CRASH_FAULT && currentState !=
BOOT_SWEEP && currentState != OTA_UPDATE && currentState != CALIBRATING) {
    baseColor = renderYellowBeacon(beaconElapsed, currentRSSI);
}

// B. 20-Second Role Check (Overrides for 1 second, offset by 5s to avoid overlap)
if (currentMillis - lastRolePing >= 20000) {
    lastRolePing = currentMillis;
}
unsigned long rolePingElapsed = currentMillis - lastRolePing;
if (rolePingElapsed > 5000 && rolePingElapsed < 6000 && currentState != CRASH_FAULT
&& currentState != BOOT_SWEEP && currentState != OTA_UPDATE && currentState !=
CALIBRATING) {
    baseColor = renderRolePing(rolePingElapsed - 5000, isHubNode);
}

// 4. RENDER TO ESP32
leds[0] = baseColor;
FastLED.show();
}

//
=====
=====
// ACTUAL SYSTEM LOGIC & HARDWARE MANAGEMENT
//
=====
=====

void manageSystemLogic(unsigned long currentMillis) {
    // Check WiFi and update RSSI every 500ms
    if (currentMillis - lastWiFiCheck >= 500) {

```

```

lastWiFiCheck = currentMillis;

if (WiFi.status() == WL_CONNECTED) {
  // Map raw ESP32 dBm (usually -90 to -30) to our 0-100 visual scale
  int rawDb = WiFi.RSSI();
  currentRSSI = map(rawDb, -90, -30, 1, 100);
  currentRSSI = constrain(currentRSSI, 1, 100);

  // Transition from connecting (Calibrating) to Connected
  if (currentState == CALIBRATING) {
    Serial.println("WiFi Connected!");
    setSystemState(CONN_ESTABLISHED);
  }
} else {
  currentRSSI = 0; // Disconnected

  // If we dropped connection while running, show interference
  if (currentState == IDLE_SCANNING || currentState == HUMAN_DETECTED ||
  currentState == AI_PROCESSING) {
    Serial.println("WiFi Connection Lost!");
    setSystemState(INTERFERENCE);
  }
}

// MOCK RADAR PIPELINE (Replace this block with your actual CSI parsing code)
// This just randomly triggers a "Human Detected" heartbeat every so often to prove it
works.
if (currentState == IDLE_SCANNING && (currentMillis - lastRadarMockEvent > 12000)) {
  if (random(10) > 6) {
    Serial.println("CSI Alert: Human Detected!");
    setSystemState(HUMAN_DETECTED);
    lastRadarMockEvent = currentMillis;
  }
}

// Return to Idle Scanning after 3 seconds of showing Human Detected
if (currentState == HUMAN_DETECTED && (currentMillis - stateStartTime > 3000)) {
  Serial.println("Clear: Returning to Scan.");
  setSystemState(IDLE_SCANNING);
}

void setSystemState(SystemState newState) {
  if (currentState != newState) {
    currentState = newState;
    stateStartTime = millis();
  }
}

```

```

}

//
=====
=====
// ANIMATION RENDERING FUNCTIONS
//
=====
=====

CRGB renderBootSweep(unsigned long currentMillis) {
    unsigned long elapsed = currentMillis - stateStartTime;
    if (elapsed < 1500) {
        uint8_t hue = (elapsed * 255) / 1500;
        return CHSV(hue, 255, 255);
    } else if (elapsed < 2000) {
        return CRGB::White;
    } else {
        // Sweep done, move to Calibrating/Connecting phase
        setSystemState(CALIBRATING);
        return CRGB::Black;
    }
}

CRGB renderCalibrating(unsigned long currentMillis) {
    // Expanding White Pulse (Used here to indicate "Seeking WiFi")
    unsigned long cycleTime = currentMillis % 2000;
    uint8_t brightness = map(cycleTime, 0, 1999, 0, 255);
    return CRGB::White.nscale8(brightness);
}

CRGB renderConnectionEstablished(unsigned long currentMillis) {
    unsigned long elapsed = currentMillis - stateStartTime;
    if (elapsed < 1500) {
        return CRGB(0, 255, 50); // Solid bright Emerald Green
    } else if (elapsed < 1600 || (elapsed > 1700 && elapsed < 1800)) {
        return CRGB(0, 255, 50); // Crisp double blink
    } else if (elapsed < 2000) {
        return CRGB::Black;
    } else {
        // Done showing success, go to Idle
        setSystemState(IDLE_SCANNING);
        return CRGB::Black;
    }
}

CRGB renderIdleScanning(unsigned long currentMillis) {
    // Deep Blue Baseline - "Breathing" effect

```

```
uint8_t breath = beatsin8(15, 30, 180);  
return CHSV(160, 255, breath);  
}
```

```
CRGB renderHumanDetected(unsigned long currentMillis) {  
    // Warm Amber Double Heartbeat  
    unsigned long beatTime = currentMillis % 1500;  
    CRGB amber = CHSV(32, 255, 255);  
  
    if (beatTime < 150) return amber;  
    else if (beatTime < 300) return CRGB::Black;  
    else if (beatTime < 450) return amber;  
    else return CRGB::Black;  
}
```

```
CRGB renderAIProcessing(unsigned long currentMillis) {  
    if (random8() > 100) return CHSV(192, 255, random8(100, 255));  
    return CRGB::Black;  
}
```

```
CRGB renderInterference(unsigned long currentMillis) {  
    uint8_t throb = beatsin8(20, 50, 255);  
    return CRGB(255, 69, 0).nscale8(throb);  
}
```

```
CRGB renderOTAUpdate(unsigned long currentMillis) {  
    uint8_t wave = beatsin8(150, 0, 255);  
    return blend(CRGB::Cyan, CRGB::Magenta, wave);  
}
```

```
CRGB renderCrashFault(unsigned long currentMillis) {  
    if (random8() > 180) return CRGB::Red;  
    return CRGB::Black;  
}
```

// --- OVERLAY RENDERERS ---

```
CRGB renderYellowBeacon(unsigned long elapsed, int rssi) {  
    CRGB yellow = CRGB::Yellow;  
  
    if (rssi == 0) {  
        if (elapsed < 100 || (elapsed > 200 && elapsed < 300)) return CRGB::Red;  
        return CRGB::Black;  
    }  
    else if (rssi >= 70) {  
        if (elapsed < 200) return yellow;  
        else if (elapsed < 600) return yellow.fadeToBlackBy(map(elapsed, 200, 600, 0, 255));  
        return CRGB::Black;  
    }
```

```

}
else if (rssi >= 30) {
    uint8_t wave = sin8(map(elapsed, 0, 1000, 0, 255));
    return yellow.nscale8(map(wave, 0, 255, 0, 128));
}
else {
    if (random8() > 128) return yellow.nscale8(25);
    return CRGB::Black;
}
}

```

```

CRGB renderRolePing(unsigned long elapsed, bool isHub) {
    if (isHub) {
        CRGB violet = CRGB::DarkViolet;
        if (elapsed < 200) return violet.nscale8(map(elapsed, 0, 200, 0, 255));
        else if (elapsed < 400) return violet.nscale8(map(elapsed, 200, 400, 255, 0));
        else if (elapsed < 600) return violet.nscale8(map(elapsed, 400, 600, 0, 255));
        else if (elapsed < 800) return violet.nscale8(map(elapsed, 600, 800, 255, 0));
        return CRGB::Black;
    } else {
        if (elapsed > 400 && elapsed < 500) return CRGB::Teal;
        return CRGB::Black;
    }
}

```